



Day 12: Making Your Project Production Ready

File Uploads, Search & Project Polish



Profile Photos

Upload & display student images



Project Polish

Professional UI & dashboard



Search

Filter records in real time



Demo Ready

Ship a production-grade app



Opening Hook

Why Do LinkedIn & Facebook Allow Profile Photos?

Think about the last time you visited a student portal, a company directory, or a social profile. The first thing your eye goes to is **the photo**. A face makes a record feel real — it builds trust and professionalism instantly.

Would your Student Management System feel complete without profile pictures? Right now it's a list of names. Today, we turn it into a professional application.

Before

Raw table of names, no faces, no identity

After

Photo + name + branch — a real student profile

Today's Mission

Build a Professional Student Management System



Profile Photo Upload

Let users upload & display images



Student Search

Filter by name, branch, or course



Better Dashboard

Stats cards, icons, responsive layout



Project Review

Validate, test, and polish before demo day



Pair Discussion — 5 Minutes

Turn to your partner. Answer these five questions out loud. If you can explain it clearly, you own it.

1

Sessions

What is the purpose of PHP sessions?
When do they start and end?

2

Login vs Auth

What is the difference between *logging in* and *authentication*?

3

CRUD

Name the four operations and give one SQL example for each.

4

Primary Keys

Why does every table need a primary key?

5

Form Validation

Why validate on the server side, not just the browser?

📌 Students should NEVER go 10 minutes without writing code. Keep this to 5 minutes max — then jump straight into Module 1.

How File Uploads Work in PHP

HTML Form

Receive \$_FILES

Save File

Store Filename

3 Things You Must Know

- `enctype="multipart/form-data"` — required on the form tag or the file never arrives
- `$_FILES['photo']` — PHP's superglobal that holds the uploaded file data
- `move_uploaded_file()` — moves the temp file to your uploads/ folder

Minimum Viable Upload Form

```
<form method="POST"
  enctype="multipart/form-data">
  <input type="file"
    name="photo">
  <button>Upload</button>
</form>
```

Live Code: PHP File Upload Handler

Type this out — do NOT copy-paste. Understanding comes from typing it yourself.

```
<?php
if ($_SERVER['REQUEST_METHOD'] == 'POST') {

    $allowed_types = ['image/jpeg', 'image/png', 'image/gif'];
    $file = $_FILES['photo'];

    // Basic validation
    if (!in_array($file['type'], $allowed_types)) {
        die("Error: Only JPG, PNG, GIF allowed.");
    }
    if ($file['size'] > 2 * 1024 * 1024) {
        die("Error: File too large (max 2MB).");
    }

    // Generate unique filename
    $ext = pathinfo($file['name'], PATHINFO_EXTENSION);
    $filename = uniqid('student_') . '.' . $ext;
    $dest = 'uploads/' . $filename;

    if (move_uploaded_file($file['tmp_name'], $dest)) {
        // Save $filename to DB ← your next step
        echo "Upload successful!";
    }
}
?>
```

 Always validate file type AND file size server-side. Never trust the browser's file picker alone.

Module 1 Mission: Upload Profile Picture

Mission Requirements

- Add a file input to the Add/Edit Student form
- Upload the image file to the `uploads/` folder
- Save the filename to a `photo` column in your `students` table
- Display the photo in the student list and profile view

Challenge

- Show a default avatar when no photo is uploaded — use a placeholder image from `assets/`
- **Bonus:** Display a Bootstrap success alert after a successful upload
- **Super Bonus:** Preview the image in the browser before the form submits using JavaScript

SQL Hint

```
ALTER TABLE students ADD photo  
VARCHAR(255);
```

Display Hint

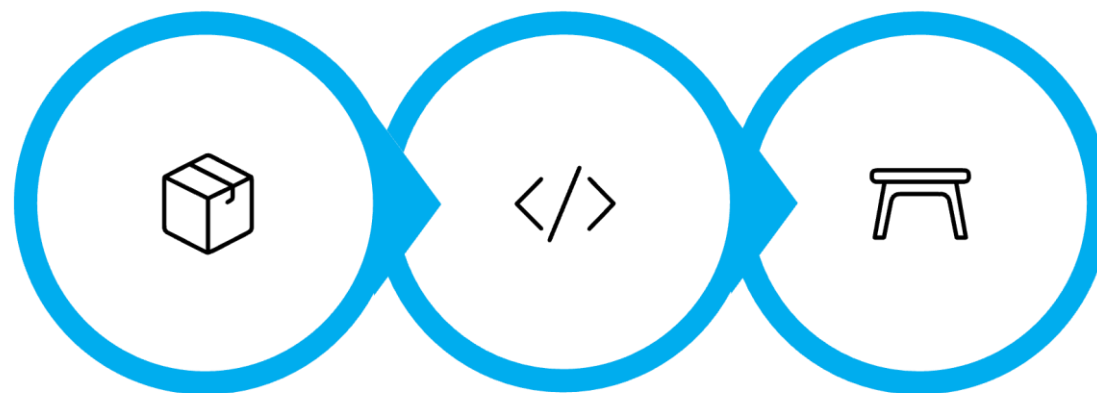
```

```

Default Hint

```
$photo = $row['photo'] ?:  
'assets/default.png';
```

How Student Search Works



Enter Query

Build SQL

Show Results

The SQL Pattern

```
SELECT * FROM students
WHERE name LIKE '%search%'
OR branch LIKE '%search%'
OR course LIKE '%search%';
```

PHP: Capture & Sanitize

```
$search = '';
if (isset($_GET['q'])) {
    $search = trim($_GET['q']);
}
$like = '%' . $search . '%';
$stmt = $pdo->prepare(
    "SELECT * FROM students
    WHERE name LIKE ?
    OR branch LIKE ?
    OR course LIKE ?"
);
$stmt->execute([$like,$like,$like]);
```


Live Code: Search Form HTML

Add this search bar above your student table. It submits via GET so the query stays in the URL — bookmarkable and shareable.

```
<!-- Search Bar (Bootstrap 5) -->
<form method="GET" action="students.php" class="d-flex mb-3">
  <input
    type="search"
    name="q"
    class="form-control me-2"
    placeholder="Search by name, branch or course..."
    value="<?= htmlspecialchars($search) ?>"
  >
  <button class="btn btn-primary" type="submit">
    <i class="bi bi-search"></i> Search
  </button>
</form>

<!-- Show result count -->
<?php if ($search): ?>
  <p class="text-muted">
    Results for: <strong><?= htmlspecialchars($search) ?></strong>
    (rowCount() ?> found)
  </p>
<?php endif; ?>
```

 Always use `htmlspecialchars()` when echoing user input back into HTML to prevent XSS attacks.

Module 2 Mission: Implement Student Search

Mission Requirements

- Add the search form above your student list table
- Write the PHP + SQL to filter results using `LIKE`
- Show a "no results found" message when the query returns zero rows
- Persist the search term in the input field after submission

Challenge

- Search across **name, branch, and course** simultaneously using a single query
- **Bonus:** Highlight the matching keyword inside the table results using `<mark>`

Empty State

When no rows returned, show: *"No students found matching your search."*

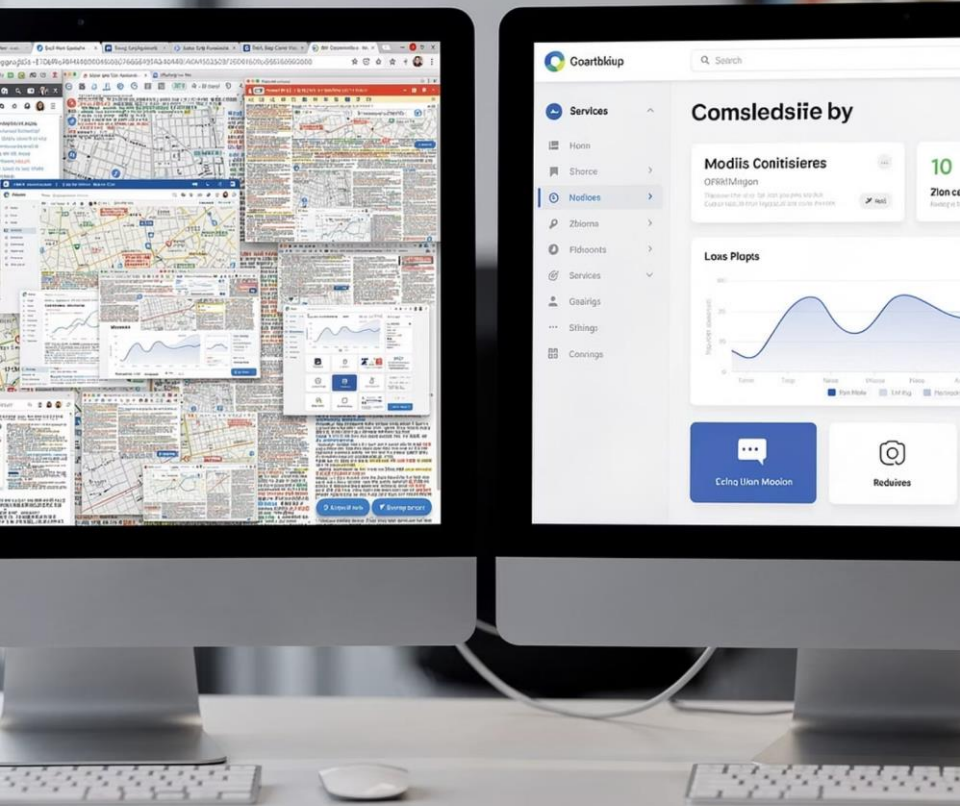
Highlight Hint

```
str_replace($search, "<mark>$search</mark>",  
$row['name'])
```

Count Hint

Use `$stmt->rowCount()` to show total results found

UI Polish: Before vs. After



✗ Before

- Plain HTML table, no Bootstrap classes
- No icons anywhere on the page
- Blue default hyperlinks as nav items
- No dashboard stats — just a raw list
- No empty-state messages

✓ After

- table-striped table-hover with avatar column
- Bootstrap Icons on every button and nav link
- Navbar with active states and dropdown
- Stats cards at top: Total Students, New This Week
- Friendly empty state with a call-to-action button

Live Code: Dashboard Stats Cards

Add this Bootstrap card row at the top of your dashboard. It's the first thing the client sees — make it count.

```
<!-- Fetch total count -->
<?php
$total = $pdo->query("SELECT COUNT(*) FROM students")->fetchColumn();
$recent = $pdo->query(
    "SELECT COUNT(*) FROM students
    WHERE created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)"
)->fetchColumn();
?>
```

```
<div class="row g-3 mb-4">
  <div class="col-md-4">
    <div class="card border-0 shadow-sm">
      <div class="card-body">
        <h6 class="text-muted">Total Students</h6>
        <h2 class="fw-bold text-primary"><?= $total ?></h2>
      </div>
    </div>
  </div>
  <div class="col-md-4">
    <div class="card border-0 shadow-sm">
      <div class="card-body">
        <h6 class="text-muted">Added This Week</h6>
        <h2 class="fw-bold text-success"><?= $recent ?></h2>
      </div>
    </div>
  </div>
</div>
```

Module 3 Mission: Improve the Student Dashboard

Mission Requirements

- Add a Bootstrap 5 navbar with active link highlighting
- Add Bootstrap Icons to all buttons: Add, Edit, Delete, Search
- Replace raw table with `table-striped table-hover`
- Add student avatar thumbnail in the table's first column
- Make the layout responsive on mobile using `col-md-*`

Challenge

- Display a "**Total Students**" stat card on the dashboard using `COUNT(*)`
- **Bonus:** Add a "Recent Registrations" widget showing the last 5 students added
- **Super Bonus:** Add a CSS fade-in animation when the dashboard loads



Bootstrap Icons CDN:

```
https://cdn.jsdelivr.net/npm/bootstrap-  
icons/font/bootstrap-icons.css
```

How to Review a Project Before Delivery

Professional developers don't just build — they **review**. Before any demo or client handoff, run through this checklist systematically.



Form Validation

Every form has server-side validation. Empty fields and wrong types show clear error messages.



No Broken Links

Click every nav item, every button. A broken link in a demo kills credibility instantly.



Empty States

What does the page look like with zero students? Show a friendly message, not a blank table.



Mobile View

Open DevTools → Toggle Device Toolbar. Does it still look good on a phone?



Session Guard

Log out and try accessing `dashboard.php` directly. You must be redirected to login.

Module 4 Mission: Review Another Team's Project

Mission — Exchange Laptops

Swap your laptop with another team. You have **10 minutes** to review their project as if you were the client receiving it for the first time.

- Test every page and every link
- Try to break the search with special characters
- Upload an oversized or wrong file type
- Attempt to access the dashboard without logging in
- Check mobile responsiveness

Challenge: Find 5 Improvements

Write down exactly **5 specific improvements** the other team should make before tomorrow's demo. Be constructive and precise:

- 1 What's broken?**
- 2 What's missing?**
- 3 What could be clearer?**

Hand back the laptop with your written feedback. The other team has 5 minutes to fix the top priority item.

Spot the Bug — 5 Broken Things

This Student Management System has **five real bugs**. Can you find them all before the reveal?

1

Broken Upload

File uploads silently fail. No error. No file saved. *What's missing from the form tag?*

2

Wrong Search

Search returns every student regardless of the keyword. *What's wrong with the SQL query?*

3

Missing Image

Student profile shows a broken image icon. *Why doesn't the fallback default image appear?*

4

Broken Navigation

The "Edit Student" link throws a 404. *What's wrong with the URL construction?*

5

Missing Session

You can access `dashboard.php` without logging in. *What single line is missing at the top of the file?*

☐ Trainer reveals each bug with the corrected code snippet after students have had 5 minutes to discuss in pairs.

30-Minute Build Sprint — Client Demo Tomorrow

This is your final window to polish your project before the big day. Work fast, work smart.

1

Fix Bugs

Address the issues found during peer review first

2

Polish UI

Icons, cards, spacing, colors — make it look professional

3

Test Everything

CRUD, login, upload, search, mobile view

4

Push to GitHub

Commit with a meaningful message before time is up

  30-minute timer is running. Your project must be demo-ready when it ends. No extensions tomorrow.



Prepare Tomorrow's 3-Minute Demo

Every team prepares a structured walkthrough. The trainer will review your demo flow before you present. Think of it as a **product pitch**, not just a code show-and-tell.

01

Project Story

What problem does it solve?
Who is it for?

02

Feature Walkthrough

Live demo: login → CRUD →
upload → search

03

Architecture

Briefly explain your file
structure and DB schema

04

Challenges & Learning

One thing that was hard and
how you solved it

Demo Day Rules

- 3 minutes max — practice to a timer
- Every team member must speak at least once
- Live demo on localhost — no slides allowed
- Be ready to answer technical questions
- Trainer will ask: *"How would you add X feature?"*

Quick-Fire Interview Questions

These are real questions you'll face in internship and job interviews. Answer in 30 seconds each — no notes.

Q1

What is the purpose of file uploads in a web application? Give one real-world example.

Q2

Why must you validate uploaded files on the server? What could go wrong if you don't?

Q3

What is the purpose of PHP sessions? How do you start and destroy one?

Q4

What is the difference between CRUD operations and authentication?



Predict the Output

```
$search = "Ali";  
$like   = "%" . $search . "%";  
echo $like;
```

What does this print? Why does it matter in a SQL LIKE query?

 ASSIGNMENT

Tonight's Assignment: Final Project Preparation

 Required — Due Before Demo

- Complete all features: CRUD + Login + Upload + Search
- Fix every bug identified during peer review today
- Improve your UI — use Bootstrap cards, icons, and responsive layout
- Add session guards to every protected page
- Push final code to GitHub with a clean commit message
- Prepare your 3-minute demo script — practice it aloud at least twice

 Bonus Points

- Add a CSS fade-in or slide-up animation to cards
- Build a better dashboard with a "Recent Registrations" table widget
- Add Bootstrap Icons throughout the entire application
- Implement client-side image preview before upload

 Your GitHub commit history IS your portfolio. Push clean, descriptive commits — future employers will read them.

Today's Achievement — You Built Something Real



✅ Profile Photos

File upload with type & size validation, stored in uploads /



✅ Student Search

SQL LIKE query filtering across name, branch, and course



✅ Better Dashboard

Stats cards, Bootstrap Icons, responsive Bootstrap 5 layout



✅ Project Review

Peer testing, bug hunting, and pre-demo quality checklist

Tomorrow you'll present your project like a professional software engineer. You're not a student anymore — you're a developer with a live, working application. Own it.

